LAB 5: STATIC CODE ANALYSIS

Music Catalog Python Program

Student	Satyanarayana Kallakuri
Course / Lab	Lab 5 – Static Code Analysis
Repository	https://github.com/SETAPESU26/L5_19_music_catalog
Submission location	PES1UG24AM257_SE / Lab 5

1. Objective

The objective of this lab was to use static analysis tools to identify and correct code-quality, security, and formatting issues in a Python music-catalog program. The tools used were Pylint, Bandit, and Flake8. The program was reviewed and updated while retaining its main catalog and playlist functionality.

2. Tools Used

Tool	Purpose
Pylint	Checks code quality, naming, common programming mistakes, and maintainability.
Bandit	Checks Python source for common security risks and unsafe coding patterns.
Flake8	Checks Python style and formatting, including whitespace and end-of-file issues.

3. Issues Identified and Corrections

Issue	Correction	Benefit
Mutable default argument in playlist function	Changed the default to `None` and initialized a new list inside the function.	Prevents playlist state from being unintentionally shared between calls.
SQL statement string formatting	Used parameterized SQL statements with placeholders.	Separates query structure from supplied values and reduces SQL-injection risk.
Unsafe pickle deserialization	Replaced pickle-based playlist serialization with JSON.	Avoids executing arbitrary object reconstruction from untrusted pickle data.
Broad exception handling	Caught the specific SQLite exception type in the total-play calculation.	Avoids hiding unrelated programming errors.
Empty music catalog	Added a check before selecting a random song ID.	Avoids an exception when no songs exist.
Naming and formatting issues	Used conventional snake_case names and corrected formatting.	Improves readability and consistency.

4. Verification Results

Tool	Observed final result
Pylint	10.00/10.00. The terminal output showed an improvement from 4.57/10 in an earlier run.
Bandit	“No issues identified” in the final report shown in the terminal.
Flake8	An end-of-file blank-line warning (W391) appeared during an earlier check. The file was edited and reports were regenerated afterward; confirm the final report is empty / the command prints no output before submission.

5. Reflection

What is the value of static analysis?
Static analysis can reveal potential bugs, unsafe patterns, and style violations before the program is executed or delivered.

How do the tools differ?
Pylint focuses on code quality and programming practices, Bandit focuses on common security problems, and Flake8 focuses on style and formatting.

What was learned?
The lab demonstrated the importance of avoiding mutable defaults, using parameterized database queries, avoiding unsafe deserialization, handling exceptions specifically, and checking edge cases.

What should be checked before final submission?
Run all three tools after the final edit, retain before-and-after reports, verify the program still runs, and confirm that the required Git commits are present in the repository.

6. Submission Checklist

• Updated Python source file: music_catalog.py

• Before-and-after reports for Pylint, Bandit, and Flake8

• Issue documentation: issues.md

• Reflection answers: reflection.md

• Git history and push to the Lab 5 folder in PES1UG24AM257_SE

Note: The results above are based on terminal output shared during the lab workflow. The required three separate code-fix commits were not demonstrated in the shared Git history; verify the commit history against the instructor's requirement before submission.